FILE: /app/backend/server.py

```
import json
import logging
import os
import re
import uuid
from datetime import datetime, timezone
from pathlib import Path
from typing import Dict, List, Literal, Optional

from dotenv import load_dotenv
from emergentintegrations.llm.chat import LlmChat, UserMessage
from fastapi import APIRouter, FastAPI, HTTPException
from motor.motor_asyncio import AsyncIOMotorClient
from pydantic import BaseModel, ConfigDict, Field
from starlette.middleware.cors import CORSMiddleware


ROOT_DIR = Path(__file__).parent
load_dotenv(ROOT_DIR / ".env")

mongo_url = os.environ["MONGO_URL"]
client = AsyncIOMotorClient(mongo_url)
db = client[os.environ["DB_NAME"]]

app = FastAPI()
api_router = APIRouter(prefix="/api")


def now_iso() -> str:
    return datetime.now(timezone.utc).isoformat()


def parse_iso_date(date_value: Optional[str]) -> Optional[datetime]:
    if not date_value:
        return None
    try:
        return datetime.fromisoformat(date_value.replace("Z", "+00:00"))
    except ValueError:
        return None


def strip_json_fence(text: str) -> str:
    matches = re.findall(r"```(?:json)?\s*(.*?)```", text, flags=re.DOTALL)
    if matches:
        return matches[0].strip()
    return text.strip()


class StatusCheck(BaseModel):
    model_config = ConfigDict(extra="ignore")

    id: str = Field(default_factory=lambda: str(uuid.uuid4()))
    client_name: str
    timestamp: datetime = Field(default_factory=lambda: datetime.now(timezone.utc))


class StatusCheckCreate(BaseModel):
    client_name: str


class TaskBase(BaseModel):
    title: str
    subject: str = "General"
    priority: Literal["low", "medium", "high"] = "medium"
    due_date: Optional[str] = None
    estimated_minutes: int = 60
    completed: bool = False


class TaskCreate(TaskBase):
    pass


class TaskUpdate(BaseModel):
    title: Optional[str] = None
    subject: Optional[str] = None
    priority: Optional[Literal["low", "medium", "high"]] = None
    due_date: Optional[str] = None
    estimated_minutes: Optional[int] = None
    completed: Optional[bool] = None


class Task(TaskBase):
```

```
    id: str
    created_at: str
    updated_at: str


class DeadlineBase(BaseModel):
    title: str
    subject: str = "General"
    due_date: str
    reminder_date: Optional[str] = None
    notes: Optional[str] = ""
    completed: bool = False


class DeadlineCreate(DeadlineBase):
    pass


class DeadlineUpdate(BaseModel):
    title: Optional[str] = None
    subject: Optional[str] = None
    due_date: Optional[str] = None
    reminder_date: Optional[str] = None
    notes: Optional[str] = None
    completed: Optional[bool] = None


class Deadline(DeadlineBase):
    id: str
    created_at: str
    updated_at: str


class FocusSessionCreate(BaseModel):
    duration_minutes: int
    started_at: Optional[str] = None
    completed: bool = True


class FocusSession(BaseModel):
    id: str
    duration_minutes: int
    started_at: str
    completed: bool
    created_at: str


class StudyPlanRequest(BaseModel):
    goals: str
    available_hours_per_day: float
    study_days: int = 7
    priorities: Optional[str] = ""


class StudyDay(BaseModel):
    day: str
    topics: List[str]
    focus_minutes: int
    notes: str


class StudyPlanResponse(BaseModel):
    id: str
    goals: str
    priorities: str
    available_hours_per_day: float
    study_days: int
    generated_schedule: List[StudyDay]
    summary: str
    raw_text: str
    created_at: str


class ReminderItem(BaseModel):
    id: str
    title: str
    due_date: str
    days_left: int
    urgency: Literal["overdue", "due_soon", "upcoming"]


class SubjectStat(BaseModel):
    subject: str
    total_tasks: int
```

```
    completed_tasks: int


class AnalyticsSummary(BaseModel):
    tasks_total: int
    tasks_completed: int
    deadlines_upcoming: int
    focus_minutes_total: int
    focus_sessions_count: int
    productivity_score: int
    subject_breakdown: List[SubjectStat] = Field(default_factory=list)


def fallback_schedule(payload: StudyPlanRequest) -> tuple[List[Dict], str]:
    topics = [part.strip() for part in re.split(r",|;", payload.goals) if part.strip()]
    if not topics:
        topics = ["General Revision"]

    plan: List[Dict] = []
    for day_index in range(payload.study_days):
        topic = topics[day_index % len(topics)]
        plan.append(
            {
                "day": f"Day {day_index + 1}",
                "topics": [topic, "Active recall"],
                "focus_minutes": int(payload.available_hours_per_day * 60),
                "notes": "Study in two focused blocks with a short revision at the end.",
            }
        )

    return plan, "Structured fallback plan generated."


def extract_schedule_source_and_summary(loaded: object) -> tuple[List[Dict], str]:
    if isinstance(loaded, dict):
        summary = loaded["summary"] if isinstance(loaded.get("summary"), str) else "AI generated plan created."
        schedule = loaded["schedule"] if isinstance(loaded.get("schedule"), list) else []
        return schedule, summary

    if isinstance(loaded, list):
        return loaded, "AI generated plan created."

    return [], "AI generated plan created."


def normalize_topics(topics: object) -> List[str]:
    if isinstance(topics, str):
        return [topics]
    if isinstance(topics, list) and topics:
        return [str(topic) for topic in topics]
    return ["General topic"]


def normalize_schedule_entries(schedule_source: List[Dict], payload: StudyPlanRequest) -> List[Dict]:
    normalized_schedule: List[Dict] = []
    default_focus_minutes = int(payload.available_hours_per_day * 60)

    for idx, item in enumerate(schedule_source[: payload.study_days]):
        if not isinstance(item, dict):
            continue

        normalized_schedule.append(
            {
                "day": str(item.get("day", f"Day {idx + 1}")),
                "topics": normalize_topics(item.get("topics")),
                "focus_minutes": int(item.get("focus_minutes", default_focus_minutes)),
                "notes": str(item.get("notes", "Review and recap for 10 minutes.")),
            }
        )

    return normalized_schedule


def pad_schedule_days(normalized_schedule: List[Dict], payload: StudyPlanRequest) -> List[Dict]:
    while len(normalized_schedule) < payload.study_days:
        next_day = len(normalized_schedule) + 1
        normalized_schedule.append(
            {
                "day": f"Day {next_day}",
                "topics": ["Revision Block"],
                "focus_minutes": int(payload.available_hours_per_day * 60),
                "notes": "Use this day for weak-topic reinforcement.",
            }
        )
```

```
252|     return normalized_schedule
253|
254|
255| def parse_plan_response(raw_text: str, payload: StudyPlanRequest) -> tuple[List[Dict], str]:
256|     cleaned = strip_json_fence(raw_text)
257|     try:
258|         loaded = json.loads(cleaned)
259|     except json.JSONDecodeError:
260|         return fallback_schedule(payload)
261|
262|     schedule_source, summary = extract_schedule_source_and_summary(loaded)
263|     normalized_schedule = normalize_schedule_entries(schedule_source, payload)
264|
265|     if not normalized_schedule:
266|         return fallback_schedule(payload)
267|
268|     return pad_schedule_days(normalized_schedule, payload), summary
269|
270|
271| def count_upcoming_deadlines(deadlines: List[Dict], now: datetime) -> int:
272|     return sum(
273|         1
274|         for deadline in deadlines
275|         if (due_date := parse_iso_date(deadline.get("due_date"))) and due_date >= now
276|     )
277|
278|
279| def build_subject_breakdown(tasks: List[Dict]) -> List[SubjectStat]:
280|     subject_map: Dict[str, Dict[str, int]] = {}
281|     for task in tasks:
282|         subject = task.get("subject", "General")
283|         if subject not in subject_map:
284|             subject_map[subject] = {"total": 0, "completed": 0}
285|         subject_map[subject]["total"] += 1
286|         if task.get("completed"):
287|             subject_map[subject]["completed"] += 1
288|
289|     subject_breakdown = [
290|         SubjectStat(subject=subject, total_tasks=stats["total"], completed_tasks=stats["completed"])
291|         for subject, stats in subject_map.items()
292|     ]
293|     subject_breakdown.sort(key=lambda item: item.total_tasks, reverse=True)
294|     return subject_breakdown
295|
296|
297| def calculate_productivity_score(tasks_total: int, tasks_completed: int, focus_minutes_total: int) -> int:
298|     completion_ratio = tasks_completed / tasks_total if tasks_total else 0
299|     focus_component = min(50, focus_minutes_total // 10)
300|     return min(100, int(completion_ratio * 50 + focus_component))
301|
302|
303| @api_router.get("/")
304| async def root():
305|     return {"message": "AI Study Planner API is running"}
306|
307|
308| @api_router.post("/status", response_model=StatusCheck)
309| async def create_status_check(input: StatusCheckCreate):
310|     status_obj = StatusCheck(**input.model_dump())
311|     doc = status_obj.model_dump()
312|     doc["timestamp"] = doc["timestamp"].isoformat()
313|     await db.status_checks.insert_one(doc)
314|     return status_obj
315|
316|
317| @api_router.get("/status", response_model=List[StatusCheck])
318| async def get_status_checks():
319|     status_checks = await db.status_checks.find({}, {"_id": 0}).to_list(1000)
320|     for check in status_checks:
321|         if isinstance(check.get("timestamp"), str):
322|             check["timestamp"] = datetime.fromisoformat(check["timestamp"])
323|     return status_checks
324|
325|
326| @api_router.post("/plans/generate", response_model=StudyPlanResponse)
327| async def generate_study_plan(payload: StudyPlanRequest):
328|     llm_key = os.environ.get("EMERGENT_LLM_KEY")
329|     if not llm_key:
330|         raise HTTPException(status_code=500, detail="Missing EMERGENT_LLM_KEY in backend environment")
331|
332|     prompt = (
333|         "Create a realistic student study plan. Return only JSON with keys: "
334|         "summary and schedule. schedule must be an array of objects with day, topics, focus_minutes, notes. "
335|         f"Goals: {payload.goals}. Priorities: {payload.priorities}. "
```

```
336|         f"Hours per day: {payload.available_hours_per_day}. Number of days: {payload.study_days}."
337|     )
338|
339|     chat = LlmChat(
340|         api_key=llm_key,
341|         session_id=f"study-plan-{uuid.uuid4()}",
342|         system_message="You are a study productivity assistant. Output strict JSON only.",
343|     ).with_model("openai", "gpt-5.2")
344|
345|     ai_response = await chat.send_message(UserMessage(text=prompt))
346|     raw_text = ai_response if isinstance(ai_response, str) else str(ai_response)
347|
348|     generated_schedule, summary = parse_plan_response(raw_text, payload)
349|
350|     plan_doc = {
351|         "id": str(uuid.uuid4()),
352|         "goals": payload.goals,
353|         "priorities": payload.priorities or "",
354|         "available_hours_per_day": payload.available_hours_per_day,
355|         "study_days": payload.study_days,
356|         "generated_schedule": generated_schedule,
357|         "summary": summary,
358|         "raw_text": raw_text,
359|         "created_at": now_iso(),
360|     }
361|
362|     await db.study_plans.insert_one(plan_doc.copy())
363|     return StudyPlanResponse(**plan_doc)
364|
365|
366| @api_router.get("/plans", response_model=List[StudyPlanResponse])
367| async def get_study_plans():
368|     plans = await db.study_plans.find({}, {"_id": 0}).sort("created_at", -1).to_list(50)
369|     return plans
370|
371|
372| @api_router.post("/tasks", response_model=Task)
373| async def create_task(payload: TaskCreate):
374|     timestamp = now_iso()
375|     task_doc = payload.model_dump()
376|     task_doc.update({"id": str(uuid.uuid4()), "created_at": timestamp, "updated_at": timestamp})
377|     await db.tasks.insert_one(task_doc.copy())
378|     return Task(**task_doc)
379|
380|
381| @api_router.get("/tasks", response_model=List[Task])
382| async def get_tasks():
383|     tasks = await db.tasks.find({}, {"_id": 0}).sort("created_at", -1).to_list(500)
384|     return tasks
385|
386|
387| @api_router.put("/tasks/{task_id}", response_model=Task)
388| async def update_task(task_id: str, payload: TaskUpdate):
389|     updates = {key: value for key, value in payload.model_dump().items() if value is not None}
390|     updates["updated_at"] = now_iso()
391|     result = await db.tasks.update_one({"id": task_id}, {"$set": updates})
392|     if result.matched_count == 0:
393|         raise HTTPException(status_code=404, detail="Task not found")
394|
395|     updated_task = await db.tasks.find_one({"id": task_id}, {"_id": 0})
396|     if not updated_task:
397|         raise HTTPException(status_code=404, detail="Task not found")
398|     return updated_task
399|
400|
401| @api_router.delete("/tasks/{task_id}")
402| async def delete_task(task_id: str):
403|     result = await db.tasks.delete_one({"id": task_id})
404|     if result.deleted_count == 0:
405|         raise HTTPException(status_code=404, detail="Task not found")
406|     return {"success": True, "message": "Task deleted"}
407|
408|
409| @api_router.post("/deadlines", response_model=Deadline)
410| async def create_deadline(payload: DeadlineCreate):
411|     timestamp = now_iso()
412|     deadline_doc = payload.model_dump()
413|     deadline_doc.update({"id": str(uuid.uuid4()), "created_at": timestamp, "updated_at": timestamp})
414|     await db.deadlines.insert_one(deadline_doc.copy())
415|     return Deadline(**deadline_doc)
416|
417|
418| @api_router.get("/deadlines", response_model=List[Deadline])
419| async def get_deadlines():
```

```
    deadlines = await db.deadlines.find({}, {"_id": 0}).sort("due_date", 1).to_list(500)
    return deadlines


@api_router.put("/deadlines/{deadline_id}", response_model=Deadline)
async def update_deadline(deadline_id: str, payload: DeadlineUpdate):
    updates = {key: value for key, value in payload.model_dump().items() if value is not None}
    updates["updated_at"] = now_iso()
    result = await db.deadlines.update_one({"id": deadline_id}, {"$set": updates})
    if result.matched_count == 0:
        raise HTTPException(status_code=404, detail="Deadline not found")

    updated_deadline = await db.deadlines.find_one({"id": deadline_id}, {"_id": 0})
    if not updated_deadline:
        raise HTTPException(status_code=404, detail="Deadline not found")
    return updated_deadline


@api_router.delete("/deadlines/{deadline_id}")
async def delete_deadline(deadline_id: str):
    result = await db.deadlines.delete_one({"id": deadline_id})
    if result.deleted_count == 0:
        raise HTTPException(status_code=404, detail="Deadline not found")
    return {"success": True, "message": "Deadline deleted"}


@api_router.get("/reminders", response_model=List[ReminderItem])
async def get_deadline_reminders():
    now = datetime.now(timezone.utc)
    deadlines = await db.deadlines.find({"completed": False}, {"_id": 0}).to_list(500)
    reminders: List[ReminderItem] = []

    for item in deadlines:
        due_date = parse_iso_date(item.get("due_date"))
        if not due_date:
            continue
        days_left = (due_date.date() - now.date()).days
        urgency: Literal["overdue", "due_soon", "upcoming"]
        if days_left < 0:
            urgency = "overdue"
        elif days_left <= 3:
            urgency = "due_soon"
        else:
            urgency = "upcoming"

        if urgency in {"overdue", "due_soon"}:
            reminders.append(
                ReminderItem(
                    id=item["id"],
                    title=item["title"],
                    due_date=item["due_date"],
                    days_left=days_left,
                    urgency=urgency,
                )
            )

    reminders.sort(key=lambda reminder: reminder.days_left)
    return reminders


@api_router.post("/focus-sessions", response_model=FocusSession)
async def create_focus_session(payload: FocusSessionCreate):
    timestamp = now_iso()
    session_doc = {
        "id": str(uuid.uuid4()),
        "duration_minutes": payload.duration_minutes,
        "started_at": payload.started_at or timestamp,
        "completed": payload.completed,
        "created_at": timestamp,
    }
    await db.focus_sessions.insert_one(session_doc.copy())
    return FocusSession(**session_doc)


@api_router.get("/focus-sessions", response_model=List[FocusSession])
async def get_focus_sessions():
    sessions = await db.focus_sessions.find({}, {"_id": 0}).sort("created_at", -1).to_list(300)
    return sessions


@api_router.get("/analytics/summary", response_model=AnalyticsSummary)
async def get_analytics_summary():
    now = datetime.now(timezone.utc)

```

```
    tasks = await db.tasks.find({}, {"_id": 0}).to_list(2000)
    deadlines = await db.deadlines.find({"completed": False}, {"_id": 0}).to_list(2000)
    focus_sessions = await db.focus_sessions.find({}, {"_id": 0}).to_list(2000)

    tasks_total = len(tasks)
    tasks_completed = len([task for task in tasks if task.get("completed")])
    focus_minutes_total = sum(int(session.get("duration_minutes", 0)) for session in focus_sessions)

    upcoming_deadlines = count_upcoming_deadlines(deadlines, now)
    subject_breakdown = build_subject_breakdown(tasks)
    productivity_score = calculate_productivity_score(tasks_total, tasks_completed, focus_minutes_total)

    return AnalyticsSummary(
        tasks_total=tasks_total,
        tasks_completed=tasks_completed,
        deadlines_upcoming=upcoming_deadlines,
        focus_minutes_total=focus_minutes_total,
        focus_sessions_count=len(focus_sessions),
        productivity_score=productivity_score,
        subject_breakdown=subject_breakdown,
    )


app.include_router(api_router)

app.add_middleware(
    CORSMiddleware,
    allow_credentials=True,
    allow_origins=os.environ.get("CORS_ORIGINS", "*").split(","),
    allow_methods=["*"],
    allow_headers=["*"],
)

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s - %(name)s - %(levelname)s - %(message)s",
)
logger = logging.getLogger(__name__)


@app.on_event("shutdown")
async def shutdown_db_client():
    client.close()
```

FILE: /app/backend/requirements.txt

```
fastapi==0.110.1
uvicorn==0.25.0
boto3>=1.34.129
requests-oauthlib>=2.0.0
cryptography>=42.0.8
python-dotenv>=1.0.1
pymongo==4.5.0
pydantic>=2.6.4
email-validator>=2.2.0
pyjwt>=2.10.1
bcrypt==4.1.3
passlib>=1.7.4
tzdata>=2024.2
motor==3.3.1
pytest>=8.0.0
black>=24.1.1
isort>=5.13.2
flake8>=7.0.0
mypy>=1.8.0
python-jose>=3.3.0
requests>=2.31.0
pandas>=2.2.0
numpy>=1.26.0
python-multipart>=0.0.9
jq>=1.6.0
typer>=0.9.0
emergentintegrations==0.1.0
```